Práctica individual de Docker Compose con cinco servicios

Frontend, Backend, MariaDB, Redis y Mailpit. Reporte de implementación y evidencias.

Asignatura	Infraestructura de Servicios
Modalidad	Individual
Estudiante	Neftalí Odín García Ramírez
Número de control	22121291
Grupo	A
Fecha	11 de octubre de 2026
Sistema asignado	Agenda de actividades (último dígito 1), entidad Actividad
Mejora individual	Fecha límite con detección de actividades vencidas y próximas a vencer
Repositorio	https://github.com/AegisOdin/practica-docker-compose

1. Descripción del proyecto

Se construyó una agenda de actividades distribuida en cinco contenedores definidos en un solo archivo `compose.yaml`. La interfaz permite crear, consultar, editar, cambiar de estado y eliminar actividades. El backend guarda la información en MariaDB, conserva el listado en Redis durante 60 segundos, cuenta cada consulta y envía un correo de prueba a Mailpit cuando se crea una actividad.

Arquitectura

```
Navegador
  └── http://localhost:8081
        └── frontend (Nginx)
              ├── archivos estáticos (HTML, CSS, JavaScript)
              └── /api/ → backend:80 (PHP 8.3 + Apache)
                          ├── db:5432      MariaDB 11.4
                          ├── redis:6379   Redis 7.4
                          └── mailpit:1025 SMTP de prueba

Mailpit UI → http://localhost:8025
```

Servicio	Imagen	Responsabilidad	Exposición
frontend	propia, desde nginx:1.27-alpine	Interfaz y proxy de /api/	Puerto 8081 del host
backend	propia, desde php:8.3-apache	API REST, caché, contador y correo	Solo red interna
db	mariadb:11.4	Persistencia de actividades	Solo red interna
redis	redis:7.4-alpine	Caché del listado y contador	Solo red interna
mailpit	axllent/mailpit:v1.21	Captura de correos de prueba	Interfaz web en 8025

> **Ajustes respecto al enunciado.** La imagen sugerida `MariaDB:16-alpine` no existe en Docker Hub, por lo que se usó `mariadb:11.4` (LTS). Para cumplir con `DB_PORT=5432`, MariaDB se inicia con `--port=5432` en lugar de su puerto habitual 3306. El volumen nombrado se monta en `/var/lib/mysql`, que es el directorio de datos real de MariaDB.

2. Proceso de implementación

Etapa 1. Preparación

Se creó la estructura de directorios pedida, el `.gitignore` (que excluye `.env`) y la plantilla `.env.example` sin contraseñas reales. El archivo `.env` local contiene contraseñas aleatorias y no se versiona. La entidad Actividad tiene título, descripción, fecha límite, estado y marcas de tiempo.

`.env.example`

```
# Copia este archivo como .env y cambia las contraseñas:
#   cp .env.example .env
```

```
# El archivo .env no se sube al repositorio (ver .gitignore).

# Zona horaria usada para calcular vencimientos
TZ=America/Mexico_City

# Puertos publicados en el host
FRONTEND_PORT=8081
MAILPIT_PORT=8025

# MariaDB
DB_PORT=5432
DB_NAME=agenda
DB_USER=agenda_app
DB_PASSWORD=cambia_esta_clave
DB_ROOT_PASSWORD=cambia_esta_clave_root

# Redis
REDIS_PORT=6379
REDIS_PASSWORD=cambia_esta_clave_redis
# Segundos que la lista permanece en caché (mínimo 30)
CACHE_TTL=60

# Correo de prueba (Mailpit)
SMTP_PORT=1025
MAIL_FROM=agenda@practica.local
MAIL_TO=estudiante@practica.local

# Días de anticipación para marcar una actividad como próxima a vencer
DIAS_AVISO=3
```

El script de inicialización define la tabla con llave primaria, restricción de unicidad (título y fecha), una restricción `CHECK` sobre la longitud del título y tres registros iniciales.

`database/init.sql`

```
-- Agenda de actividades: esquema y datos iniciales.
-- MariaDB ejecuta este archivo una sola vez, cuando el volumen de datos está vacío.

SET NAMES utf8mb4;

CREATE TABLE IF NOT EXISTS actividades (
    id             INT UNSIGNED NOT NULL AUTO_INCREMENT,
    titulo         VARCHAR(120) NOT NULL,
    descripcion    VARCHAR(500) NOT NULL DEFAULT '',
    fecha_limite   DATE         NOT NULL,
    estado         ENUM('pendiente', 'en_progreso', 'completada') NOT NULL DEFAULT 'pendiente',
    creado_en      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    CONSTRAINT uq_actividad_titulo_fecha UNIQUE (titulo, fecha_limite),
    CONSTRAINT chk_actividad_titulo CHECK (CHAR_LENGTH(TRIM(titulo)) >= 3),
    INDEX idx_actividad_fecha (fecha_limite)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

-- Las fechas se calculan respecto al día de la inicialización para que la agenda
-- muestre desde el primer arranque una actividad vencida, una próxima y una lejana.
INSERT INTO actividades (titulo, descripcion, fecha_limite, estado) VALUES
    ('Entregar práctica de Docker Compose',
     'Subir el repositorio y el PDF de evidencias de Infraestructura de Servicios.',
     CURDATE() + INTERVAL 2 DAY, 'en_progreso'),
    ('Repasar redes y volúmenes de Docker',
     'Leer los apuntes de redes bridge, DNS interno y volúmenes nombrados.',
     CURDATE() - INTERVAL 1 DAY, 'pendiente'),
    ('Exposición del proyecto final',
```

```
    'Preparar las diapositivas y ensayar la demostración en vivo.',
    CURDATE() + INTERVAL 15 DAY, 'pendiente');
```

Etapa 2. Base de datos y Redis

En `compose.yaml` se declaró `db` con su volumen `mariadb_data`, el script de inicialización montado en modo lectura y un health check con `healthcheck.sh`. Redis usa persistencia AOF (`--appendonly yes`), contraseña y un health check con `redis-cli ping`.

`compose.yaml`

```
name: agenda-actividades

services:
  frontend:
    build: ./frontend
    restart: unless-stopped
    ports:
      - "${FRONTEND_PORT:-8081}:80"
    depends_on:
      backend:
        condition: service_healthy
    networks:
      - app_network

  backend:
    build: ./backend
    restart: unless-stopped
    expose:
      - "80"
    environment:
      TZ: ${TZ:-America/Mexico_City}
      DB_HOST: db
      DB_PORT: ${DB_PORT:-5432}
      DB_NAME: ${DB_NAME:?Define DB_NAME en .env}
      DB_USER: ${DB_USER:?Define DB_USER en .env}
      DB_PASSWORD: ${DB_PASSWORD:?Define DB_PASSWORD en .env}
      REDIS_HOST: redis
      REDIS_PORT: ${REDIS_PORT:-6379}
      REDIS_PASSWORD: ${REDIS_PASSWORD:?Define REDIS_PASSWORD en .env}
      CACHE_TTL: ${CACHE_TTL:-60}
      SMTP_HOST: mailpit
      SMTP_PORT: ${SMTP_PORT:-1025}
      MAIL_FROM: ${MAIL_FROM:-agenda@practica.local}
      MAIL_TO: ${MAIL_TO:-estudiante@practica.local}
      DIAS_AVISO: ${DIAS_AVISO:-3}
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
      mailpit:
        condition: service_started
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://127.0.0.1/health"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    networks:
      - app_network

  db:
    image: mariadb:11.4
```

```
    restart: unless-stopped
    command:
      - --port=${DB_PORT:-5432}
      - --character-set-server=utf8mb4
      - --collation-server=utf8mb4_unicode_ci
    expose:
      - "${DB_PORT:-5432}"
    environment:
      TZ: ${TZ:-America/Mexico_City}
      MARIADB_DATABASE: ${DB_NAME:?Define DB_NAME en .env}
      MARIADB_USER: ${DB_USER:?Define DB_USER en .env}
      MARIADB_PASSWORD: ${DB_PASSWORD:?Define DB_PASSWORD en .env}
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?Define DB_ROOT_PASSWORD en .env}
    volumes:
      - mariadb_data:/var/lib/mysql
      - ./database/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
    networks:
      - app_network

  redis:
    image: redis:7.4-alpine
    restart: unless-stopped
    command: ["redis-server", "--appendonly", "yes", "--requirepass", "${REDIS_PASSWORD:?Define
REDIS_PASSWORD en .env}"]
    expose:
      - "6379"
    environment:
      # Permite usar redis-cli dentro del contenedor sin escribir la contraseña.
      REDISCLI_AUTH: ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD-SHELL", "redis-cli ping | grep -q PONG"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks:
      - app_network

  mailpit:
    image: axllent/mailpit:v1.21
    restart: unless-stopped
    ports:
      - "${MAILPIT_PORT:-8025}:8025"
    expose:
      - "1025"
    environment:
      TZ: ${TZ:-America/Mexico_City}
    networks:
      - app_network

networks:
  app_network:
    driver: bridge

volumes:
  mariadb_data:
  redis_data:
```

Etapa 3. Backend

La imagen del backend parte de `php:8.3-apache` e instala `pdo_mysql` para MariaDB y la extensión `redis`. Apache entrega todas las rutas a `index.php`, que funciona como enrutador.

`backend/Dockerfile`

```
FROM php:8.3-apache

# Extensiones para MariaDB (pdo_mysql) y Redis (phpredis)
RUN docker-php-ext-install pdo_mysql \
    && pecl install redis-6.1.0 \
    && docker-php-ext-enable redis

# Toda ruta que no sea un archivo se entrega a index.php, que hace de enrutador
RUN { \
        echo 'ServerName backend'; \
        echo 'ServerTokens Prod'; \
        echo 'ServerSignature Off'; \
        echo 'FallbackResource /index.php'; \
    } > /etc/apache2/conf-available/api.conf \
    && a2enconf api \
    && mv "$PHP_INI_DIR/php.ini-production" "$PHP_INI_DIR/php.ini" \
    && echo 'expose_php = Off' > "$PHP_INI_DIR/conf.d/api.ini"

COPY src/ /var/www/html/

EXPOSE 80
```

Método	Ruta	Descripción	Códigos
GET	/api/health	Conectividad con MariaDB y Redis	200, 503
GET	/api/actividades	Listado con caché y contador	200
GET	/api/actividades/{id}	Una actividad	200, 404
POST	/api/actividades	Crea, invalida la caché y envía el correo	201, 400, 409, 422
PUT	/api/actividades/{id}	Modifica campos e invalida la caché	200, 400, 404, 409, 422
DELETE	/api/actividades/{id}	Elimina e invalida la caché	200, 404

El backend se conecta con los nombres de servicio `db`, `redis` y `mailpit`, recibidos en las variables `DB_HOST`, `REDIS_HOST` y `SMTP_HOST`. Nunca usa `localhost` para llegar a otro contenedor. El listado se guarda en la clave `actividades:todas` con TTL de 60 segundos, cada consulta incrementa `contador_consultas` y toda alta, cambio o baja borra la clave de la caché.

Etapa 4. Mailpit

Al crear una actividad, el backend abre una conexión SMTP con `mailpit:1025` y envía un mensaje con el folio, título, descripción, fecha límite y estado. Mailpit es una herramienta de desarrollo: captura los mensajes para revisarlos en `http://localhost:8025`, pero no entrega correo real a ningún buzón.

Etapa 5. Frontend y proxy

Nginx sirve los archivos estáticos y reenvía `/api/` al backend. El JavaScript usa la URL relativa `/api`, de modo que el navegador solo habla con el frontend.

`frontend/Dockerfile`

```
FROM nginx:1.27-alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY src/ /usr/share/nginx/html/

EXPOSE 80

HEALTHCHECK --interval=15s --timeout=3s --retries=3 \
    CMD wget -q -O /dev/null http://127.0.0.1/ || exit 1
```

`frontend/nginx.conf`

```
server {
    listen 80;
    server_name _;
    server_tokens off;

    root /usr/share/nginx/html;
    index index.html;

    # DNS interno de Docker. Al usar una variable en proxy_pass, Nginx vuelve a
    # resolver "backend" en cada intervalo en lugar de guardar la IP del arranque,
    # así el proxy sigue funcionando si el backend se recrea o se escala.
    resolver 127.0.0.11 valid=10s ipv6=off;

    location /api/ {
        set $backend http://backend:80;
        rewrite ^/api/(.*)$ /$1 break;
        proxy_pass $backend;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_connect_timeout 5s;
        proxy_read_timeout 15s;
    }

    location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-cache";
    }
}
```

Etapa 6. Integración

El proyecto completo se levanta con `docker compose up -d --build`. El backend espera a que `db` y `redis` estén saludables, y el frontend espera al backend.

```
$ curl -s -i http://localhost:8081/api/health
HTTP/1.1 200 OK
Server: nginx
Date: Mon, 12 Oct 2026 03:52:31 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 78
Connection: keep-alive

{"estado":"ok","mariadb":"ok","redis":"ok","hora":"2026-10-11T21:52:31-06:00"}
```

3. Pruebas obligatorias

Las capturas de terminal muestran la salida real de cada comando ejecutado sobre el proyecto en funcionamiento. Las capturas de la interfaz y de Mailpit se tomaron en el navegador sobre la aplicación en ejecución.

3.1 Validación YAML

```
$ docker compose config --quiet && echo 'compose.yaml válido'
compose.yaml válido
```

3.2 Estado: cinco servicios activos, db y redis saludables

```
$ docker compose ps
SERVICE    IMAGE                          STATUS                   PORTS
backend    agenda-actividades-backend     Up 31 minutes (healthy)  80/tcp
db         mariadb:11.4                   Up 31 minutes (healthy)  3306/tcp, 5432/tcp
frontend   agenda-actividades-frontend    Up 32 minutes (healthy)  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
mailpit    axllent/mailpit:v1.21          Up 33 minutes (healthy)  1025/tcp, 1110/tcp, 0.0.0.0:8025->8025/tcp, [::]:8025->8025/tcp
redis      redis:7.4-alpine               Up 33 minutes (healthy)  6379/tcp
```

```
$ docker compose images
CONTAINER                      REPOSITORY                 TAG          PLATFORM       IMAGE ID       SIZE      CREATED
agenda-actividades-backend-1   agenda-actividades-backend  latest       linux/amd64    ad09d41cc926   178MB     About a minute ago
agenda-actividades-db-1        mariadb                    11.4         linux/amd64    1292844148b3   106MB     4 minutes ago
agenda-actividades-frontend-1  agenda-actividades-frontend latest       linux/amd64    dea47b02369f   21MB      About a minute ago
agenda-actividades-mailpit-1   axllent/mailpit            v1.21        linux/amd64    81370195cd4a   12.6MB    5 minutes ago
agenda-actividades-redis-1     redis                      7.4-alpine   linux/amd64    858f009f9709   16.3MB    5 minutes ago
```

3.3 CRUD desde la interfaz



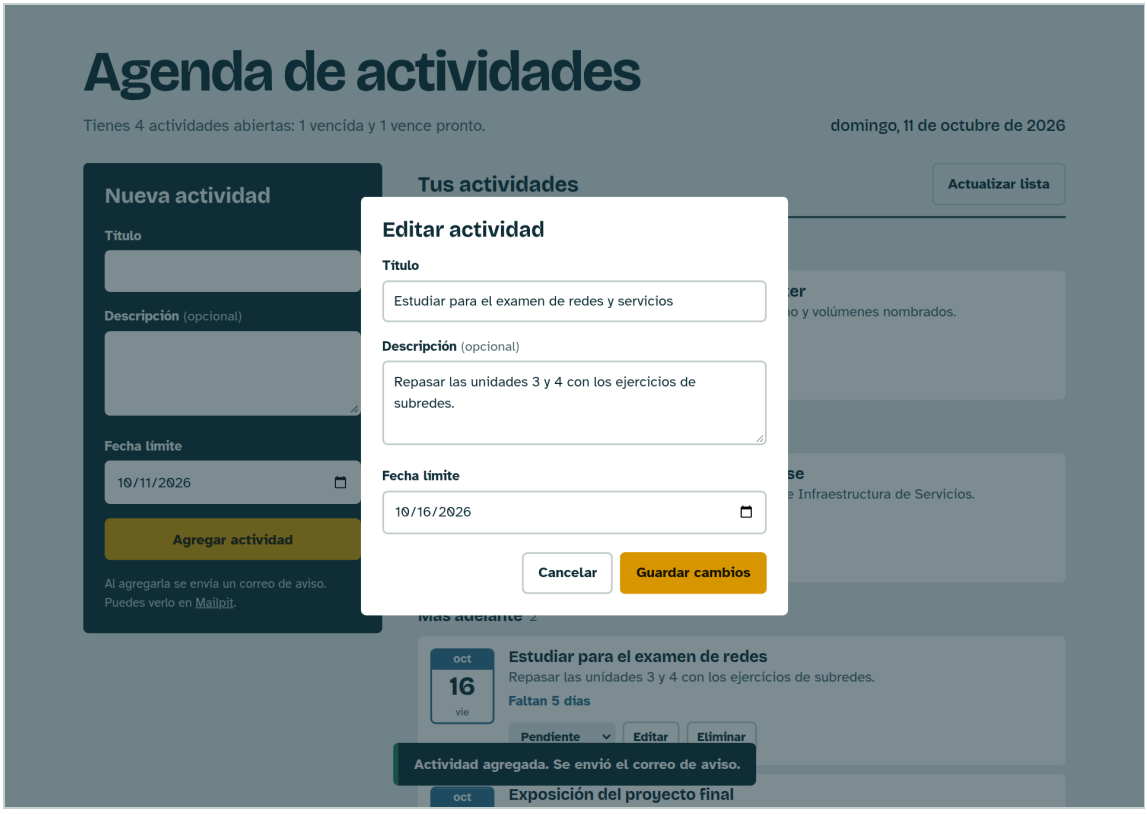
Listado inicial con los tres registros de init.sql, agrupados por vencimiento.



Formulario de alta con los datos de una actividad nueva.



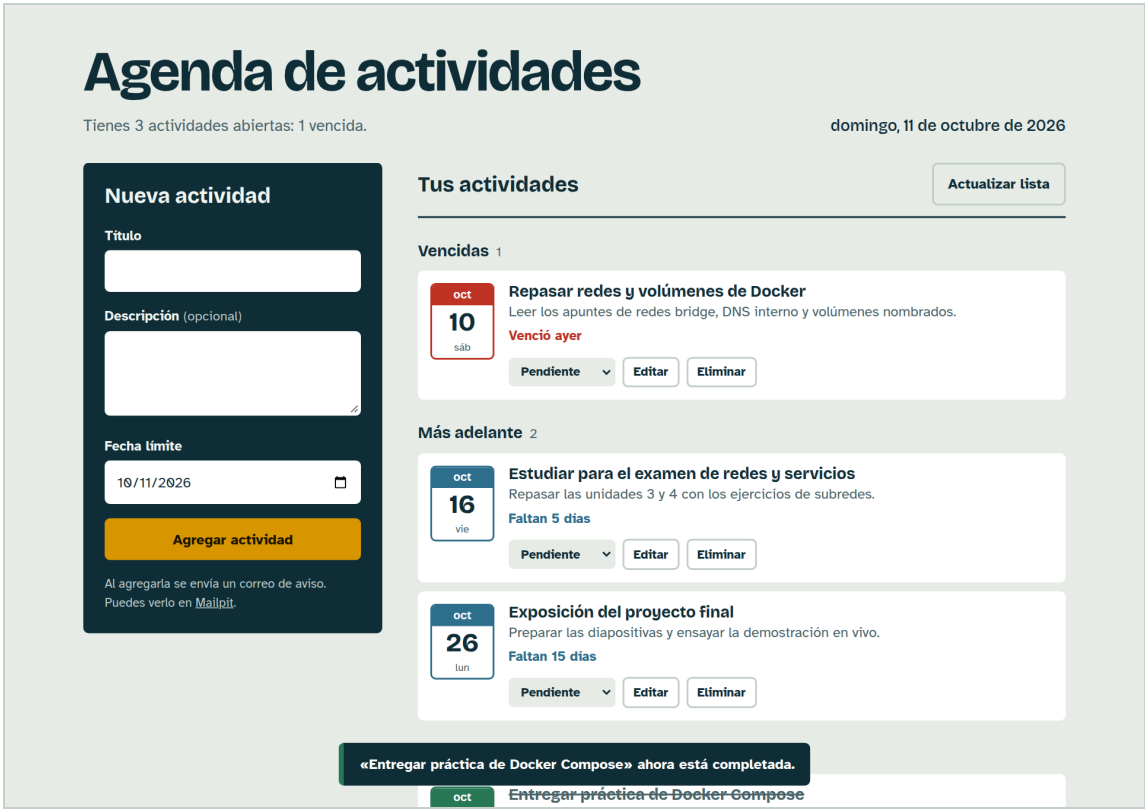
Actividad creada: mensaje de éxito y la actividad en el grupo «Más adelante».



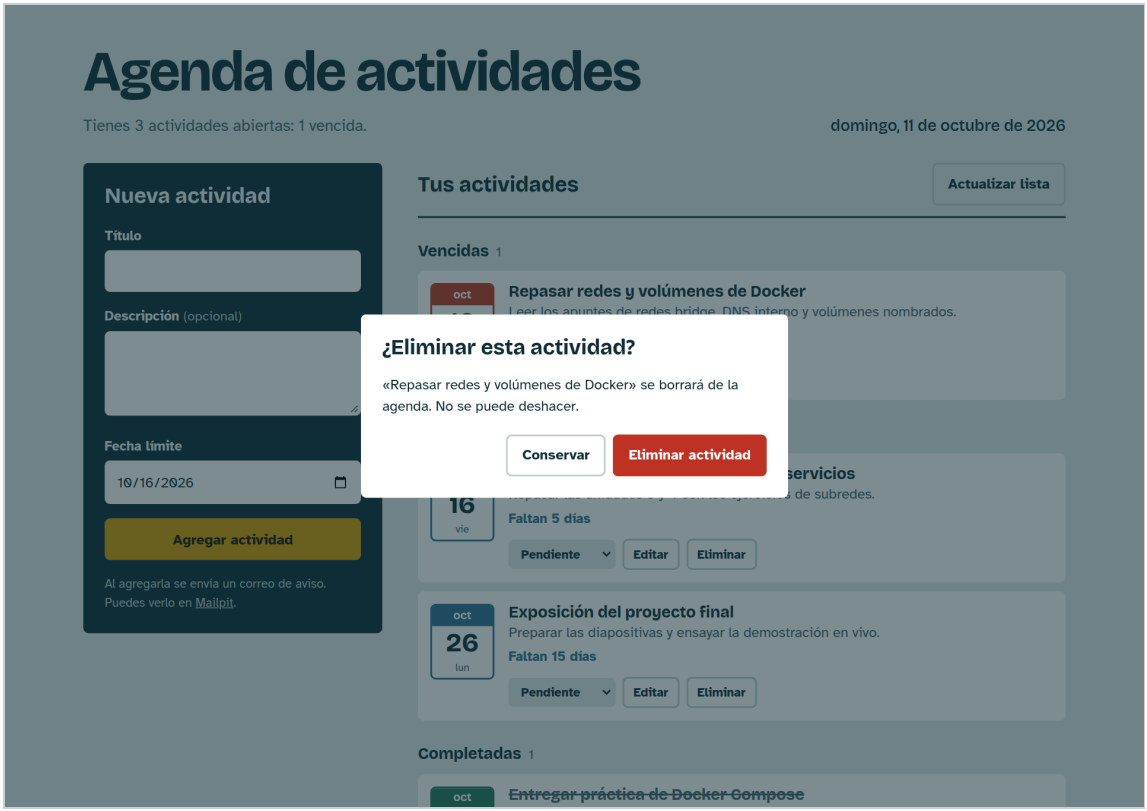
Edición del título de la actividad.



Cambios guardados y visibles en el listado.



Cambio de estado a «Completada» desde el selector.



Eliminación con confirmación del usuario.

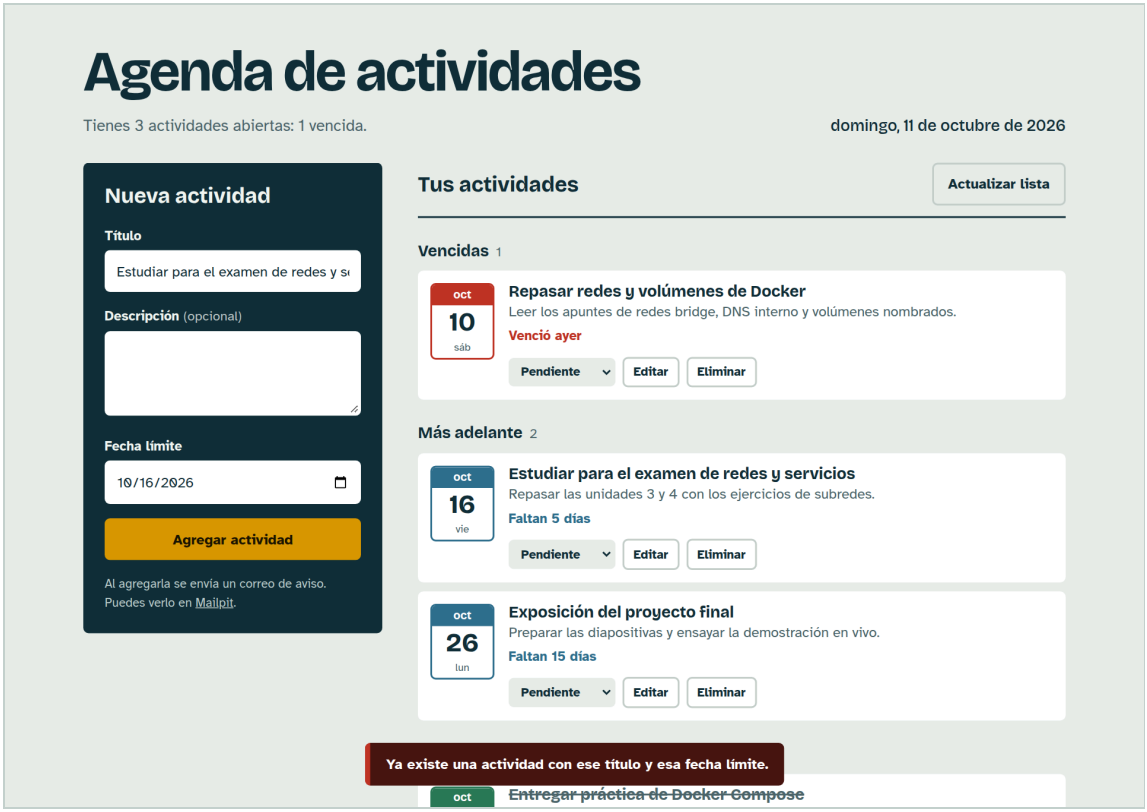


Actividad eliminada del listado.

Mensajes de error



Validación del formulario: título demasiado corto.



Error 409 del backend al intentar registrar una actividad duplicada.



La misma interfaz en el ancho de un teléfono. Al pie se indica si la lista vino de Redis o de MariaDB y el número de consulta.

Datos en MariaDB después de las operaciones anteriores:

```
$ docker compose exec -T db sh -c 'mariadb -t -u"$MARIADB_USER" -p"$MARIADB_PASSWORD" "$MARIADB_DAT
ABASE" -e "SELECT id, titulo, fecha_limite, estado FROM actividades"'
+----+-------------------------------------------+--------------+------------+
| id | titulo                                    | fecha_limite | estado     |
+----+-------------------------------------------+--------------+------------+
|  1 | Entregar práctica de Docker Compose       | 2026-10-13   | completada |
|  3 | Exposición del proyecto final             | 2026-10-26   | pendiente  |
|  4 | Estudiar para el examen de redes y servicios | 2026-10-16   | pendiente  |
+----+-------------------------------------------+--------------+------------+
```

3.4 Caché: la segunda consulta se atiende desde Redis

```
$ curl -s -D- -o /dev/null http://localhost:8081/api/actividades | grep -i x-cache
X-Cache: MISS

$ curl -s -D- -o /dev/null http://localhost:8081/api/actividades | grep -i x-cache
X-Cache: HIT

$ docker compose exec redis redis-cli TTL actividades:todas
60

$ docker compose logs --no-log-prefix backend | grep -o '\[cache\].*' | tail -4
[cache] HIT GET /actividades (consulta #6, origen redis), referer: http://localhost:8081/
[cache] INVALIDADA (cambio en la actividad 3)
[cache] MISS GET /actividades (consulta #7, origen mariadb)
[cache] HIT GET /actividades (consulta #8, origen redis)
```

3.5 Invalidación: el dato nuevo aparece de inmediato

```
$ curl -s -X POST http://localhost:8081/api/actividades -H 'Content-Type: application/json' -d '{"t
itulo":"Comprar cable de red","fecha_limite":"2026-10-14"}' | head -c 120; echo ...
{"mensaje":"Actividad creada.","correo_enviado":true,"actividad":{"id":6,"titulo":"Comprar cable de red","descripcion":"...

$ curl -s -D- -o /dev/null http://localhost:8081/api/actividades | grep -i x-cache
X-Cache: MISS

$ curl -s http://localhost:8081/api/actividades | grep -o '"titulo":"Comprar cable de red"'
"titulo":"Comprar cable de red"
```

3.6 Contador y claves de Redis

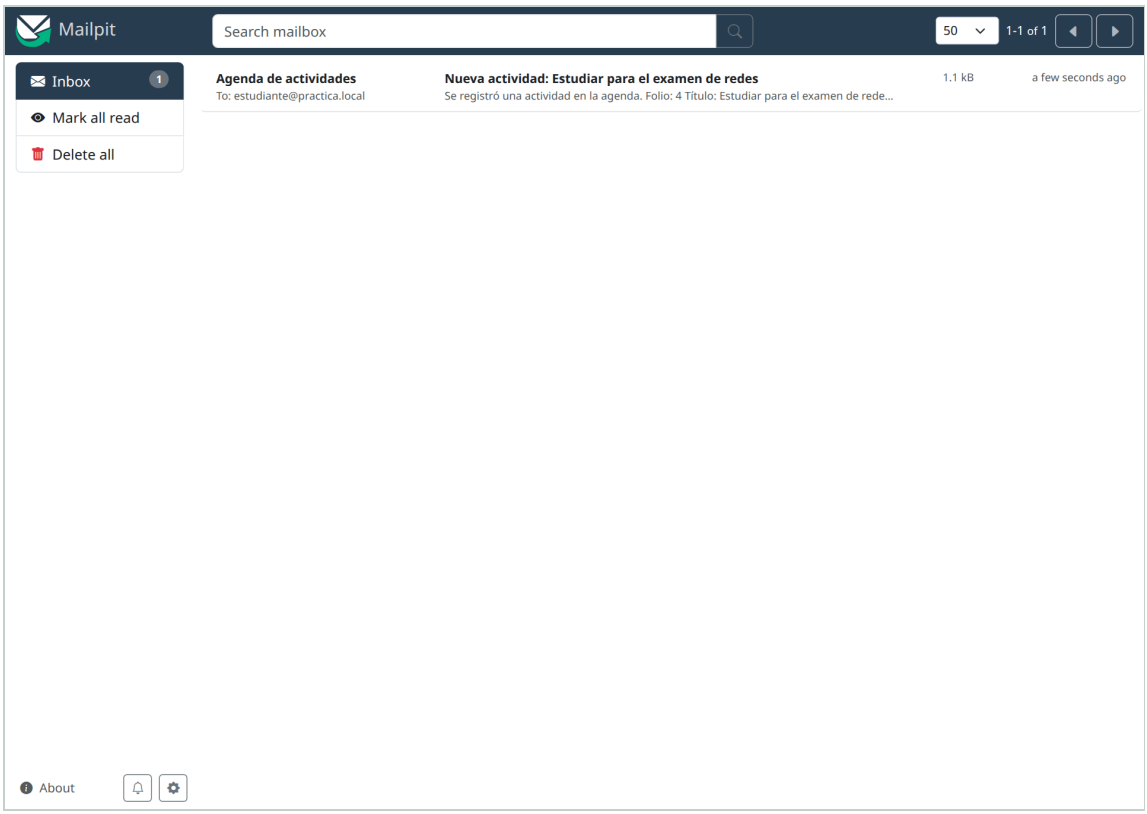
```
$ docker compose exec redis redis-cli KEYS '*'
actividades:todas
contador_consultas

$ docker compose exec redis redis-cli GET contador_consultas
8

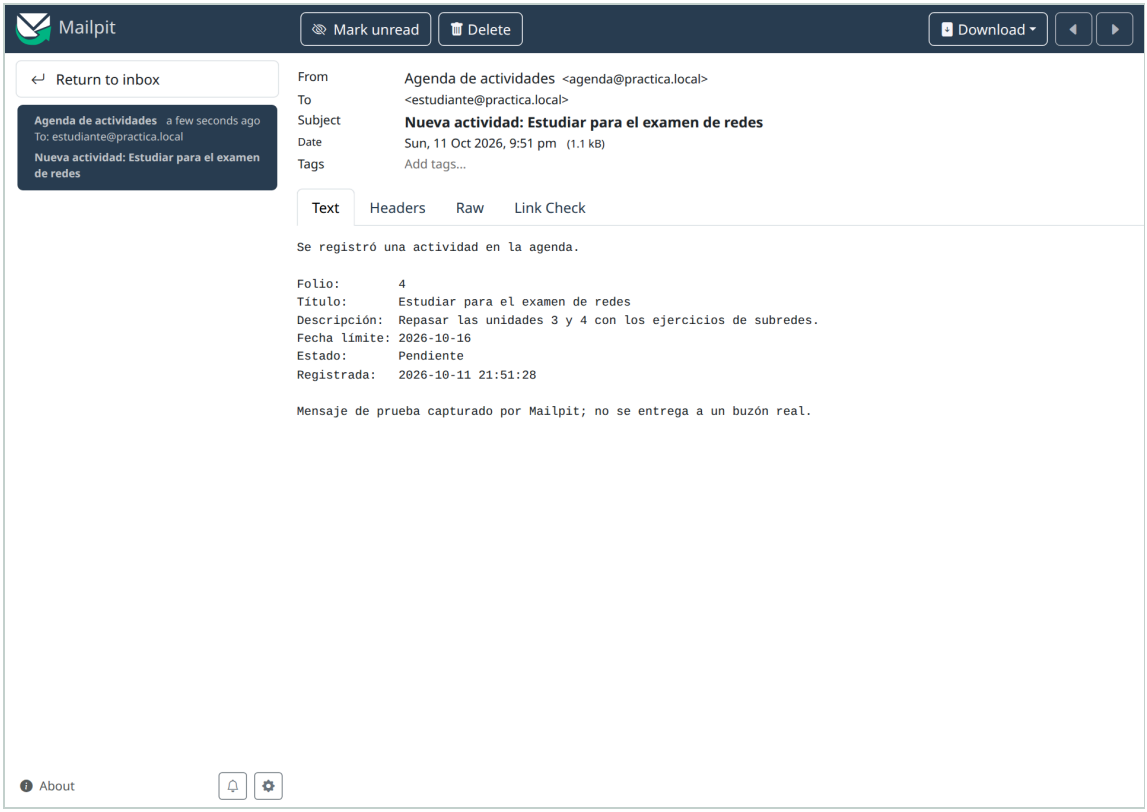
$ for i in 1 2 3; do curl -s -o /dev/null http://localhost:8081/api/actividades; done

$ docker compose exec redis redis-cli GET contador_consultas
11
```

3.7 Correo en Mailpit



Bandeja de Mailpit en http://localhost:8025 con la notificación recibida.



Asunto, destinatario y contenido del correo generado al crear la actividad.

```
$ curl -s http://localhost:8025/api/v1/messages | python3 -c "import sys,json; [print(m['Created'][:19], '|', m['To'][0]['Address'], '|', m['Subject']) for m in json.load(sys.stdin)['messages']]"
2026-10-11T21:52:31 | estudiante@practica.local | Nueva actividad: Comprar cable de red
2026-10-11T21:51:28 | estudiante@practica.local | Nueva actividad: Estudiar para el examen de redes
```

3.8 Persistencia: se recrea el contenedor de MariaDB sin eliminar el volumen

```
$ docker compose exec -T db sh -c 'mariadb -t -u"$MARIADB_USER" -p"$MARIADB_PASSWORD" "$MARIADB_DAT
ABASE" -e "SELECT id, titulo, fecha_limite, estado FROM actividades"'
+----+-----------------------------------------+--------------+------------+
| id | titulo                                  | fecha_limite | estado     |
+----+-----------------------------------------+--------------+------------+
|  1 | Entregar práctica de Docker Compose     | 2026-10-13   | completada |
|  3 | Exposición del proyecto final           | 2026-10-26   | pendiente  |
|  4 | Estudiar para el examen de redes y servicios | 2026-10-16   | pendiente  |
|  6 | Comprar cable de red                    | 2026-10-14   | pendiente  |
+----+-----------------------------------------+--------------+------------+

$ docker compose rm -sf db
 Container agenda-actividades-db-1  Stopping
 Container agenda-actividades-db-1  Stopped
Going to remove agenda-actividades-db-1
 Container agenda-actividades-db-1  Removing
 Container agenda-actividades-db-1  Removed

$ docker compose up -d db
 Container agenda-actividades-db-1  Creating
 Container agenda-actividades-db-1  Created
 Container agenda-actividades-db-1  Starting
 Container agenda-actividades-db-1  Started

$ docker compose ps db --format 'table {{.Service}}\t{{.Status}}'
SERVICE   STATUS
db        Up 12 seconds (healthy)

$ docker compose exec -T db sh -c 'mariadb -t -u"$MARIADB_USER" -p"$MARIADB_PASSWORD" "$MARIADB_DAT
ABASE" -e "SELECT id, titulo, fecha_limite, estado FROM actividades"'
+----+-----------------------------------------+--------------+------------+
| id | titulo                                  | fecha_limite | estado     |
+----+-----------------------------------------+--------------+------------+
|  1 | Entregar práctica de Docker Compose     | 2026-10-13   | completada |
|  3 | Exposición del proyecto final           | 2026-10-26   | pendiente  |
|  4 | Estudiar para el examen de redes y servicios | 2026-10-16   | pendiente  |
|  6 | Comprar cable de red                    | 2026-10-14   | pendiente  |
+----+-----------------------------------------+--------------+------------+

$ docker volume inspect agenda-actividades_mariadb_data --format '{{.Name}}  {{.Mountpoint}}  cread
o {{.CreatedAt}}'
agenda-actividades_mariadb_data  /var/lib/docker/volumes/agenda-actividades_mariadb_data/_data  creado 2026-10-11T21:51:03-06:00
```

3.9 Aislamiento: db, redis y backend sin puertos publicados

En la columna PORTS solo `frontend` y `mailpit` tienen una asignación hacia el host (`0.0.0.0:8081` y `0.0.0.0:8025`). Los demás puertos aparecen sin flecha: son internos a la red. El segundo comando confirma que el host solo escucha en 8081 y 8025.

```
$ docker compose ps --format 'table {{.Service}}\t{{.Ports}}'
SERVICE    PORTS
backend    80/tcp
db         3306/tcp, 5432/tcp
frontend   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
mailpit    1025/tcp, 1110/tcp, 0.0.0.0:8025->8025/tcp, [::]:8025->8025/tcp
redis      6379/tcp

$ ss -ltn | grep -E ':(8081|8025|5432|3306|6379|1025) ' | awk '{print $4}'
0.0.0.0:8025
0.0.0.0:8081
[::]:8025
[::]:8081
```

3.10 Recuperación: se detiene y se reinicia el backend

```
$ docker compose stop backend
 Container agenda-actividades-backend-1  Stopping
 Container agenda-actividades-backend-1  Stopped

$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://localhost:8081/api/health
HTTP 502

$ docker compose start backend
 Container agenda-actividades-db-1  Waiting
 Container agenda-actividades-redis-1  Waiting
 Container agenda-actividades-db-1  Healthy
 Container agenda-actividades-redis-1  Healthy
 Container agenda-actividades-backend-1  Starting
 Container agenda-actividades-backend-1  Started

$ curl -s -w '  HTTP %{http_code}\n' http://localhost:8081/api/health
{"estado":"ok","mariadb":"ok","redis":"ok","hora":"2026-10-11T21:52:53-06:00"}  HTTP 200

$ curl -s -o /dev/null -w 'listado: HTTP %{http_code}\n' http://localhost:8081/api/actividades
listado: HTTP 200
```

4. Cómo se resuelven los nombres db, redis, mailpit y backend

Compose crea la red `agenda-actividades_app_network`, de tipo bridge, y conecta a ella los cinco contenedores. En las redes definidas por el usuario, Docker ofrece a cada contenedor un servidor DNS interno en `127.0.0.11`. Ese servidor registra el nombre de cada servicio de Compose como alias de red y responde con la IP que el contenedor tiene en ese momento.

Por eso el backend se conecta a `db`, `redis` y `mailpit` sin conocer ninguna dirección IP, y Nginx reenvía `/api/` a `backend`. Si un contenedor se recrea y cambia de IP, el nombre sigue apuntando al contenedor nuevo. `localhost` no sirve porque dentro de un contenedor se refiere al propio contenedor.

```
$ docker network inspect agenda-actividades_app_network --format '{{.Name}} ({{.Driver}}, {{(index
.IPAM.Config 0).Subnet}}){{range .Containers}}{{println}}  {{.Name}}  {{.IPv4Address}}{{end}}'
agenda-actividades_app_network (bridge, 172.18.0.0/16)
  agenda-actividades-redis-1  172.18.0.3/16
  agenda-actividades-db-1  172.18.0.2/16
  agenda-actividades-backend-1  172.18.0.5/16
  agenda-actividades-mailpit-1  172.18.0.4/16
  agenda-actividades-frontend-1  172.18.0.6/16

$ docker compose exec backend getent hosts db redis mailpit
172.18.0.2      db
172.18.0.3      redis
172.18.0.4      mailpit

$ docker compose exec frontend getent hosts backend
172.18.0.5        backend  backend
```

5. Mejora individual: fecha límite

Cada actividad tiene una fecha de vencimiento obligatoria. El backend calcula los días restantes y clasifica cada actividad; el frontend las agrupa y colorea la hoja de calendario de cada una.

Clasificación	Condición	En la interfaz
vencida	La fecha ya pasó y no está completada	Grupo «Vencidas», hoja roja
proxima	Vence hoy o en los próximos 3 días	Grupo «Vencen pronto», hoja amarilla
en_tiempo	Faltan más de 3 días	Grupo «Más adelante», hoja azul
completada	Estado completada	Grupo «Completadas», hoja verde y título tachado

El cálculo se hace después de leer la caché, así que la clasificación corresponde siempre al día actual.

6. Preguntas de reflexión

¿Por qué el backend debe usar db:5432 y no localhost para conectarse a MariaDB?

Cada contenedor tiene su propia pila de red, así que `localhost` dentro del backend es el propio backend, donde no hay base de datos. `db` es el nombre del servicio, que el DNS interno traduce a la IP del contenedor de MariaDB.

¿Cuál es la diferencia entre ports y expose en Docker Compose?

`ports` publica un puerto del contenedor en el host y lo hace accesible desde fuera de Docker. `expose` solo documenta el puerto para los servicios de la misma red; no abre nada en el host.

¿Qué información permanece cuando se elimina un contenedor y por qué?

Permanece lo guardado en volúmenes: los datos de MariaDB y el AOF de Redis. Los volúmenes tienen un ciclo de vida independiente del contenedor; la capa de escritura del contenedor sí se pierde.

¿Qué problema resuelve el health check antes de iniciar el backend?

Que un contenedor esté iniciado no significa que el servicio ya acepte conexiones. Con `condition: service_healthy` el backend arranca hasta que MariaDB y Redis responden, y se evitan errores de conexión en el arranque.

¿Cuándo puede una caché devolver información desactualizada y cómo lo evitaste?

Cuando los datos cambian en MariaDB mientras la copia en Redis sigue vigente. Toda alta, cambio o baja borra la clave `actividades:todas`, y además la clave expira sola a los 60 segundos.

¿Por qué Mailpit no debe considerarse un servidor de correo de producción?

Porque no entrega los mensajes, solo los captura. Tampoco exige autenticación ni cifrado por defecto y su almacenamiento es temporal.

¿Qué servicio escalarías primero y qué cambio harías para evitar conflictos de puertos?

El backend, con `docker compose up -d --scale backend=3`. Puede escalarse porque no publica puertos en el host ni tiene `container_name`. Un servicio con puerto fijo publicado no admite réplicas: habría que quitar el puerto fijo o usar un rango y colocar un balanceador delante.

¿Qué riesgos existirían si MariaDB y Redis se publicaran hacia Internet?

Ataques de fuerza bruta sobre las contraseñas, explotación de vulnerabilidades del servicio y robo, alteración o borrado de datos. Redis en particular se diseñó para redes de confianza.

7. Repositorio y lista de cotejo

El repositorio está publicado en https://github.com/AegisOdin/practica-docker-compose y contiene el código fuente, los dos Dockerfiles, `compose.yaml`, `database/init.sql`, `.env.example`, `.gitignore` y el `README.md` con las instrucciones para reproducir el proyecto desde cero.

```
proyecto-garcia-neftali/
├── compose.yaml
├── .env.example
├── .gitignore
├── README.md
├── database/init.sql
├── backend/   Dockerfile, src/ (index.php, db.php, cache.php, mailer.php)
├── frontend/  Dockerfile, nginx.conf, src/ (index.html, styles.css, app.js)
└── docs/      este reporte y las capturas
```

Punto de la lista de cotejo	Cumplido
compose.yaml pasa docker compose config	Sí (3.1)

Los cinco servicios aparecen en docker compose ps	Sí (3.2)
No se usa container_name	Sí
El backend usa db, redis y mailpit como nombres de host	Sí (sección 4)
MariaDB conserva información al recrear su contenedor	Sí (3.8)
Redis muestra claves, TTL y contador	Sí (3.4 y 3.6)
Mailpit recibe una notificación al crear un registro	Sí (3.7)
El frontend consume /api/ por medio de Nginx	Sí
.env no está versionado	Sí
El README permite reproducir el proyecto desde cero	Sí